Postprocessing Robustness Evaluation This notebook evaluates reconstruction-based postprocessing under synthetic distortions and reports final robustness metrics and figures.

In [1]:
```
import os, sys
from pathlib import Path

# If opened from notebooks/, jump to repo root so relative paths work.
ROOT = Path.cwd()
if not (ROOT / "src").exists():
    ROOT = ROOT.parent
os.chdir(ROOT)
print("ROOT:", Path.cwd())

sys.path.append(str(ROOT))
```

```
ROOT: c:\Users\dhruv\temporal_cityscapes
```

Build validation loader and on-the-fly corruption utilities

In [2]:
```
from torch.utils.data import DataLoader
from src.dataset_cityscapes import CityscapesTrainId
import torch
import torch.nn.functional as F

# Evaluation defaults
DATA_ROOT = "data"
SIZE_HW = (512, 1024)
BATCH_SIZE = 4
NUM_WORKERS = 0  # Wrapper/corruption loaders are usually more stable this way on W
BLUR_K = 9
NOISE_SIGMA = 0.14
BRIGHT_FACTOR = 1.25
JPEG_QUALITY = 25
DISTORTIONS = ["blur", "gaussian_noise", "brightness_shift", "jpeg_compression"]

# Clean validation loader
ds_val_clean = CityscapesTrainId(
    root=DATA_ROOT,
    split="val",
    size_hw=SIZE_HW
)

loader_val_clean = DataLoader(
    ds_val_clean,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=True
)

print("val samples:", len(ds_val_clean), "val batches:", len(loader_val_clean))

def jpeg_like_corruption(x, quality=25):
    """Approximate JPEG artifacts with quantization + blockiness in normalized tens
```

```
    x01 = torch.clamp((x + 3.0) / 6.0, 0.0, 1.0)
    q_step = max(2, int((100 - quality) / 4))
    xq = torch.round((x01 * 255.0) / q_step) * q_step / 255.0

    h, w = x.shape[-2:]
    h8 = max(8, (h // 8) * 8)
    w8 = max(8, (w // 8) * 8)
    x_small = F.interpolate(xq, size=(h8 // 2, w8 // 2), mode="bilinear", align_cor
    x_block = F.interpolate(x_small, size=(h8, w8), mode="nearest")
    if h8 != h or w8 != w:
        x_block = F.interpolate(x_block, size=(h, w), mode="nearest")

    return torch.clamp(x_block * 6.0 - 3.0, -3.0, 3.0)

def apply_corruption(x, mode):
    """On-the-fly corruption transforms for robustness evaluation."""
    if mode == "blur":
        return F.avg_pool2d(x, kernel_size=BLUR_K, stride=1, padding=BLUR_K // 2)
    if mode == "gaussian_noise":
        return torch.clamp(x + NOISE_SIGMA * torch.randn_like(x), -3.0, 3.0)
    if mode == "brightness_shift":
        return torch.clamp(x * BRIGHT_FACTOR, -3.0, 3.0)
    if mode == "jpeg_compression":
        return jpeg_like_corruption(x, quality=JPEG_QUALITY)
    raise ValueError(f"Unknown corruption mode: {mode}")

def reverse_corruption(x, mode):
    """Lightweight reverse transforms used as auxiliary reconstruction priors."""
    if mode == "blur":
        sharp = x + 0.8 * (x - F.avg_pool2d(x, kernel_size=5, stride=1, padding=2))
        return torch.clamp(sharp, -3.0, 3.0)
    if mode == "gaussian_noise":
        return F.avg_pool2d(x, kernel_size=3, stride=1, padding=1)
    if mode == "brightness_shift":
        return torch.clamp(x / max(BRIGHT_FACTOR, 1e-6), -3.0, 3.0)
    if mode == "jpeg_compression":
        return F.avg_pool2d(x, kernel_size=3, stride=1, padding=1)
    raise ValueError(f"Unknown corruption mode: {mode}")

# Quick sanity check
x0, y0, meta0 = next(iter(loader_val_clean))
x_blur = apply_corruption(x0, "blur")
x_jpeg = apply_corruption(x0, "jpeg_compression")
x_rev = reverse_corruption(x_blur, "blur")
print("x clean:", x0.shape, "x blur:", x_blur.shape, "x jpeg:", x_jpeg.shape, "x re
```

```
val samples: 500 val batches: 125
x clean: torch.Size([4, 3, 512, 1024]) x blur: torch.Size([4, 3, 512, 1024]) x jpeg:
torch.Size([4, 3, 512, 1024]) x rev: torch.Size([4, 3, 512, 1024]) y: torch.Size([4,
512, 1024])
```

Prediction + confidence helper

In [3]:

```
import torch

IGNORE_INDEX = 255
```

```python
NUM_CLASSES = 19

@torch.inference_mode()
def pred_and_confidence(model, x):
    out = model(x)["out"]                    # (B,C,H,W)
    probs = torch.softmax(out, dim=1)        # (B,C,H,W)
    conf, pred = probs.max(dim=1)            # (B,H,W), (B,H,W)
    return pred, conf
```

In [4]:

```python
import torchvision

# Self-contained model load so this notebook does not depend on previous kernel sta
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
ckpt_path = Path("weights") / "deeplabv3_cityscapes_baseline.pt"

model = torchvision.models.segmentation.deeplabv3_resnet50(
    weights=None,
    weights_backbone=None,
    num_classes=NUM_CLASSES,
    aux_loss=True,
)

state = torch.load(ckpt_path, map_location="cpu")
if isinstance(state, dict):
    if "model_state_dict" in state:
        sd = state["model_state_dict"]
    elif "state_dict" in state:
        sd = state["state_dict"]
    elif "model" in state and isinstance(state["model"], dict):
        sd = state["model"]
    else:
        sd = state
else:
    raise ValueError("Checkpoint format not supported")

model_sd = model.state_dict()
sd_filtered = {}
skipped = []
for k, v in sd.items():
    k2 = k[7:] if k.startswith("module.") else k
    if k2 in model_sd and model_sd[k2].shape == v.shape:
        sd_filtered[k2] = v
    else:
        skipped.append(k2)

incomp = model.load_state_dict(sd_filtered, strict=False)
model = model.to(device).eval()

print("Loaded:", ckpt_path)
print("device:", device)
print("loaded tensors:", len(sd_filtered), "skipped:", len(skipped))
print("missing keys:", len(incomp.missing_keys), "unexpected keys:", len(incomp.une
print("example skipped keys:", skipped[:5])
```

```
Loaded: weights\deeplabv3_cityscapes_baseline.pt
device: cuda
loaded tensors: 368 skipped: 2
missing keys: 2 unexpected keys: 0
example skipped keys: ['aux_classifier.4.weight', 'aux_classifier.4.bias']
```

Reconstruction baseline 1: nearest confident label fill

In [5]:
```
import numpy as np
from scipy.ndimage import distance_transform_edt

def recon_nearest_confident(pred, conf, tau):
    """Legacy nearest-confident fill (kept for quick ablations)."""
    pred_np = pred.detach().cpu().numpy()
    conf_np = conf.detach().cpu().numpy()

    hi = conf_np >= tau
    if hi.sum() == 0:
        return pred

    _, inds = distance_transform_edt(~hi, return_indices=True)
    rr, cc = inds[0], inds[1]

    filled = pred_np.copy()
    filled[~hi] = pred_np[rr[~hi], cc[~hi]]

    return torch.from_numpy(filled).to(pred.device).long()
```

Mask-aware reconstruction: region fill + confidence-weighted logit diffusion

In [6]:
```
from scipy.ndimage import binary_dilation, distance_transform_edt, label

def neighbor_majority_from_pred(pred, num_classes=19, kernel_size=3):
    """Batched local majority label using one-hot + avg_pool2d."""
    one_hot = F.one_hot(pred.long(), num_classes=num_classes).permute(0, 3, 1, 2).f
    counts = F.avg_pool2d(one_hot, kernel_size=kernel_size, stride=1, padding=kerne
    return counts.argmax(dim=1).long()

def tune_tau_for_rejection(conf_map, tau_base, min_reject=0.05, target_low=0.10, ta
    """Adapt tau per image to keep rejection in a useful range."""
    conf_flat = conf_map.detach().reshape(-1)
    tau_img = float(tau_base)
    rej = float((conf_map < tau_img).float().mean().item())

    if rej < min_reject:
        tau_img = max(tau_img, float(torch.quantile(conf_flat, 0.20).item()))
        rej = float((conf_map < tau_img).float().mean().item())
        if rej < target_low:
            tau_img = max(tau_img, float(torch.quantile(conf_flat, target_high).ite
            rej = float((conf_map < tau_img).float().mean().item())

    return float(min(max(tau_img, 0.0), 0.999)), rej

def repair_mask_from_pred_conf(pred, conf, tau, num_classes=19, majority_kernel=3,
    """Primary mask: low confidence; optional tiny boundary expansion for continuit
```

```
    neigh = neighbor_majority_from_pred(pred, num_classes=num_classes, kernel_size=
    repair = (conf < tau) | (((pred != neigh) & (conf < (tau + 0.02))))

    if repair.float().mean().item() < 0.01:
        q_tau = torch.quantile(conf.reshape(conf.shape[0], -1), 0.15, dim=1).view(-
        repair = repair | (conf < q_tau)

    if dilation > 0:
        rep_np = repair.detach().cpu().numpy().astype(bool)
        struct = np.ones((3, 3), dtype=bool)
        for b in range(rep_np.shape[0]):
            m = rep_np[b]
            for _ in range(dilation):
                m = binary_dilation(m, structure=struct)
            rep_np[b] = m
        repair = torch.from_numpy(rep_np).to(pred.device)

    return repair.bool()

def recon_nearest_confident_masked(pred, conf, repair_mask, tau):
    """Fill repair_mask pixels from nearest trusted source (never from the same mas
    pred_np = pred.detach().cpu().numpy()
    conf_np = conf.detach().cpu().numpy()
    rep_np = repair_mask.detach().cpu().numpy().astype(bool)

    source = (~rep_np) & (conf_np >= tau)
    if source.sum() == 0:
        source = ~rep_np
    if source.sum() == 0:
        return pred

    _, inds = distance_transform_edt(~source, return_indices=True)
    rr, cc = inds[0], inds[1]

    filled = pred_np.copy()
    filled[rep_np] = pred_np[rr[rep_np], cc[rep_np]]
    return torch.from_numpy(filled).to(pred.device).long()

def recon_regionfill_majority(pred, conf, repair_mask, tau, num_classes=19):
    """Component fill on repair_mask using confident boundary majority labels."""
    pred_np = pred.detach().cpu().numpy()
    conf_np = conf.detach().cpu().numpy()
    rep_np = repair_mask.detach().cpu().numpy().astype(bool)

    if rep_np.sum() == 0:
        return pred

    comp, n_comp = label(rep_np, structure=np.ones((3, 3), dtype=bool))
    filled = pred_np.copy()
    structure = np.ones((3, 3), dtype=bool)

    for cid in range(1, n_comp + 1):
        region = comp == cid
        boundary = binary_dilation(region, structure=structure) & (~region)
        boundary_hi = boundary & (conf_np >= tau)
        if not boundary_hi.any():
```

```
            boundary_hi = boundary
        if not boundary_hi.any():
            continue
        labels = pred_np[boundary_hi].astype(np.int64)
        binc = np.bincount(labels, minlength=num_classes)
        maj = int(binc.argmax())
        filled[region] = maj

    return torch.from_numpy(filled).to(pred.device).long()

def recon_logit_diffusion_masked(
    logits, tau, kernel_size=5, gamma=2, iters=3, num_classes=19, majority_kernel=3
):
    """Confidence-weighted diffusion only on repair pixels."""
    cur = logits.clone()
    for _ in range(iters):
        probs = torch.softmax(cur, dim=1)
        conf, pred = probs.max(dim=1)
        repair_mask = repair_mask_from_pred_conf(
            pred, conf, tau, num_classes=num_classes, majority_kernel=majority_kern
        )
        if not repair_mask.any():
            break

        smoothed = F.avg_pool2d(
            cur, kernel_size=kernel_size, stride=1, padding=kernel_size // 2
)
        w = conf.clamp(0.0, 1.0).pow(gamma).unsqueeze(1)
        blended = w * cur + (1.0 - w) * smoothed
        cur = torch.where(repair_mask.unsqueeze(1), blended, cur)

    return cur.argmax(dim=1).long()

def enforce_effective_change(
    pred_base, pred_new, probs_base, repair_mask, aux_pred=None, aux_conf=None, min
):
    """Conservative replacement with auxiliary-confidence fusion on uncertain pixel
    probs2 = torch.topk(probs_base, k=2, dim=0).values
    margin = probs2[0] - probs2[1]

    base_conf = probs_base.gather(0, pred_base.unsqueeze(0)).squeeze(0)
    cand_conf = probs_base.gather(0, pred_new.unsqueeze(0)).squeeze(0)

    # Stage 1: keep reconstruction edits only if they are not clearly worse than ba
    accept = repair_mask & (cand_conf >= (base_conf - 0.005))
    out = torch.where(accept, pred_new, pred_base)

    changed = float((out != pred_base).float().mean().item())
    forced = False

    if aux_pred is not None and aux_conf is not None and changed < min_change:
        forced = True
        # Stage 2: trust auxiliary prediction where it is confidently better.
        aux_better = repair_mask & (aux_conf >= (base_conf + 0.02))
        out = torch.where(aux_better, aux_pred.long(), out)
        changed = float((out != pred_base).float().mean().item())
```

```
        # Stage 3: in near-tie regions, allow mild auxiliary takeover.
        if changed < min_change:
            near_tie = repair_mask & (margin <= 0.06) & (aux_conf >= (base_conf - 0
            out = torch.where(near_tie, aux_pred.long(), out)
            changed = float((out != pred_base).float().mean().item())

    return out.long(), changed, forced
```

mIoU (self-contained, no dependency on “evaluate()”)

In [7]:
```
def batch_miou(pred, y, num_classes=19, ignore_index=255):
    """pred, y are (B,H,W) torch long tensors."""
    pred = pred.view(-1)
    y = y.view(-1)

    valid = y != ignore_index
    pred = pred[valid]
    y = y[valid]

    ious = []
    for c in range(num_classes):
        y_c = y == c
        if y_c.sum() == 0:
            continue
        p_c = pred == c
        inter = (p_c & y_c).sum().item()
        union = (p_c | y_c).sum().item()
        ious.append(inter / (union + 1e-8))
    if len(ious) == 0:
        return 0.0
    return float(np.mean(ious))

def maybe_log_progress(step, total_steps, values, prefix=""):
    if step == 1 or step % 10 == 0 or step == total_steps:
        msg = ", ".join([f"{k}={v:.4f}" for k, v in values.items()])
        print(f"{prefix}[{step}/{total_steps}] {msg}")
```

Sweep diffusion hyperparameters, then run final full evaluation

In [8]:
```
# 1) FAST SWEEP (subset)
SWEEP_MAX_BATCHES = 25  # first 25 batches == 100 images at batch_size=4
TAU_LIST = [0.60, 0.75, 0.85, 0.90, 0.95]
KERNEL_LIST = [3, 5]
GAMMA_LIST = [1, 2, 4]
ITERS_LIST = [1, 3, 5]

configs = [(t, k, g, it) for t in TAU_LIST for k in KERNEL_LIST for g in GAMMA_LIST
distortions = DISTORTIONS

def eval_subset_for_sweep(dist_name):
    total = min(len(loader_val_clean), SWEEP_MAX_BATCHES)
    base_scores = []
    diff_scores = {cfg: [] for cfg in configs}
```

```
    with torch.inference_mode():
        for step, (x0, y0, _) in enumerate(loader_val_clean, start=1):
            if step > total:
                break
            x0 = x0.to(device, non_blocking=True)
            y0 = y0.to(device, non_blocking=True)
            x1 = apply_corruption(x0, dist_name)
            x_aux = reverse_corruption(x1, dist_name)

            logits = model(x1)["out"]
            probs = torch.softmax(logits, dim=1)
            conf, pred_base = probs.max(dim=1)

            logits_aux = model(x_aux)["out"]
            probs_aux = torch.softmax(logits_aux, dim=1)
            conf_aux, pred_aux = probs_aux.max(dim=1)

            base_batch = batch_miou(pred_base, y0, NUM_CLASSES, IGNORE_INDEX)
            base_scores.append(base_batch)

            for cfg in configs:
                tau, ksz, gam, iters = cfg
                pred_list = []
                for i in range(x1.size(0)):
                    tau_i, rej_i = tune_tau_for_rejection(
                        conf[i], tau, min_reject=0.05, target_low=0.10, target_high
                    )
                    repair_i = repair_mask_from_pred_conf(
                        pred_base[i:i+1], conf[i:i+1], tau_i, num_classes=NUM_CLASS
                    )[0]
                    pred_i = recon_logit_diffusion_masked(
                        logits[i:i+1], tau=tau_i, kernel_size=ksz, gamma=gam, iters
                        num_classes=NUM_CLASSES, majority_kernel=3
                    )[0]
                    pred_i, ch_i, _ = enforce_effective_change(
                        pred_base[i], pred_i, probs[i], repair_i, aux_pred=pred_aux
                    )
                    pred_list.append(pred_i)
                    print(
                        f"[sweep-{dist_name}] img={step}:{i} cfg=(tau={tau:.2f},k={
                        f"tau_eff={tau_i:.4f} rejection_rate={rej_i:.3%} changed_pi
                    )
                pred_diff = torch.stack(pred_list, dim=0)
                diff_scores[cfg].append(batch_miou(pred_diff, y0, NUM_CLASSES, IGNO

            if step == 1 or step % 5 == 0 or step == total:
                print(f"[sweep-{dist_name}] {step}/{total} baseline={np.mean(base_s

    return float(np.mean(base_scores)), {cfg: float(np.mean(v)) for cfg, v in diff_

sweep_dist_results = {}
for dname in distortions:
    print(f"Starting sweep on {dname}...")
    base_mean, diff_means = eval_subset_for_sweep(dname)
    sweep_dist_results[dname] = {"baseline": base_mean, "diff": diff_means}
```

```python
rows = []
for cfg in configs:
    tau, ksz, gam, iters = cfg
    row = {"tau": tau, "kernel": ksz, "gamma": gam, "iters": iters}
    base_vals = []
    diff_vals = []
    deltas = []
    for dname in distortions:
        b = sweep_dist_results[dname]["baseline"]
        d = sweep_dist_results[dname]["diff"][cfg]
        delta = d - b
        row[f"baseline_{dname}"] = b
        row[f"diff_{dname}"] = d
        row[f"delta_{dname}"] = delta
        base_vals.append(b)
        diff_vals.append(d)
        deltas.append(delta)
    row["avg_baseline"] = float(np.mean(base_vals))
    row["avg_diff"] = float(np.mean(diff_vals))
    row["avg_delta"] = float(np.mean(deltas))
    rows.append(row)

# Best config is based on average improvement over baseline.
rows_sorted = sorted(rows, key=lambda r: r["avg_delta"], reverse=True)
best = rows_sorted[0]
BEST_CFG = {
    "tau": best["tau"],
    "kernel": best["kernel"],
    "gamma": best["gamma"],
    "iters": best["iters"],
}

print("\nTop 10 sweep configs (by avg_delta):")
for r in rows_sorted[:10]:
    print(
        f"tau={r['tau']:.2f}, k={r['kernel']}, g={r['gamma']}, it={r['iters']} | "
        f"avg_base={r['avg_baseline']:.4f}, avg_diff={r['avg_diff']:.4f}, avg_delta
    )

sweep_out = Path("outputs") / "reconstruction_sweep.txt"
sweep_out.parent.mkdir(parents=True, exist_ok=True)
with open(sweep_out, "w", encoding="utf-8") as f:
    header = ["tau", "kernel", "gamma", "iters"]
    for dname in distortions:
        header += [f"baseline_{dname}", f"diff_{dname}", f"delta_{dname}"]
    header += ["avg_baseline", "avg_diff", "avg_delta"]
    f.write(" | ".join(header) + "\n")
    for r in rows_sorted:
        vals = [
            f"{r['tau']:.2f}", str(r['kernel']), str(r['gamma']), str(r['iters'])
        ]
        for dname in distortions:
            vals += [
                f"{r[f'baseline_{dname}']:.6f}",
                f"{r[f'diff_{dname}']:.6f}",
                f"{r[f'delta_{dname}']:+.6f}",
```

```
            ]
        vals += [f"{r['avg_baseline']:.6f}", f"{r['avg_diff']:.6f}", f"{r['avg_delt
        f.write(" | ".join(vals) + "\n")
print("saved:", sweep_out)
print("BEST_CFG:", BEST_CFG)

# 2) FINAL FULL EVAL
method_order = ["baseline", "nn", "regionfill", "diffusion"]
final_results = {}

for dname in distortions:
    print(f"\nFinal eval: {dname}")
    scores = {k: [] for k in method_order}
    total_steps = len(loader_val_clean)

    with torch.inference_mode():
        for step, (x0, y0, _) in enumerate(loader_val_clean, start=1):
            x0 = x0.to(device, non_blocking=True)
            y0 = y0.to(device, non_blocking=True)
            x1 = apply_corruption(x0, dname)
            x_aux = reverse_corruption(x1, dname)

            logits = model(x1)["out"]
            probs = torch.softmax(logits, dim=1)
            conf, pred_base = probs.max(dim=1)

            logits_aux = model(x_aux)["out"]
            probs_aux = torch.softmax(logits_aux, dim=1)
            conf_aux, pred_aux = probs_aux.max(dim=1)

            pred_nn_list = []
            pred_rf_list = []
            pred_df_list = []
            rej_rates = []
            tau_vals = []
            forced_counts = {"nn": 0, "regionfill": 0, "diffusion": 0}

            for i in range(x1.size(0)):
                tau_i, rej_i = tune_tau_for_rejection(
                    conf[i], BEST_CFG["tau"], min_reject=0.05, target_low=0.10, tar
                )
                tau_vals.append(tau_i)
                rej_rates.append(rej_i)

                repair_i = repair_mask_from_pred_conf(
                    pred_base[i:i+1], conf[i:i+1], tau_i, num_classes=NUM_CLASSES,
                )[0]

                nn_i = recon_nearest_confident_masked(pred_base[i], conf[i], repair
                nn_i, ch_nn_i, forced_nn = enforce_effective_change(
                    pred_base[i], nn_i, probs[i], repair_i, aux_pred=pred_aux[i], a
                )
                forced_counts["nn"] += int(forced_nn)
                pred_nn_list.append(nn_i)

                rf_i = recon_regionfill_majority(
```

```
                pred_base[i], conf[i], repair_i, tau_i, num_classes=NUM_CLASSES
            )
            rf_i, ch_rf_i, forced_rf = enforce_effective_change(
                pred_base[i], rf_i, probs[i], repair_i, aux_pred=pred_aux[i], a
            )
            forced_counts["regionfill"] += int(forced_rf)
            pred_rf_list.append(rf_i)

            df_i = recon_logit_diffusion_masked(
                logits[i:i+1], tau=tau_i, kernel_size=BEST_CFG["kernel"],
                gamma=BEST_CFG["gamma"], iters=BEST_CFG["iters"],
                num_classes=NUM_CLASSES, majority_kernel=3
            )[0]
            df_i, ch_df_i, forced_df = enforce_effective_change(
                pred_base[i], df_i, probs[i], repair_i, aux_pred=pred_aux[i], a
            )
            forced_counts["diffusion"] += int(forced_df)
            pred_df_list.append(df_i)

            print(
                f"[{dname}] img={step}:{i} tau={tau_i:.4f} rejection_rate={rej_
                f"changed_pixel_ratio(nn/rf/df)=({ch_nn_i:.3%}/{ch_rf_i:.3%}/{c
            )

        pred_nn = torch.stack(pred_nn_list, dim=0)
        pred_rf = torch.stack(pred_rf_list, dim=0)
        pred_df = torch.stack(pred_df_list, dim=0)

        ch_nn = float((pred_nn != pred_base).float().mean().item())
        ch_rf = float((pred_rf != pred_base).float().mean().item())
        ch_df = float((pred_df != pred_base).float().mean().item())

        miou_base = batch_miou(pred_base, y0, NUM_CLASSES, IGNORE_INDEX)
        miou_nn = batch_miou(pred_nn, y0, NUM_CLASSES, IGNORE_INDEX)
        miou_rf = batch_miou(pred_rf, y0, NUM_CLASSES, IGNORE_INDEX)
        miou_df = batch_miou(pred_df, y0, NUM_CLASSES, IGNORE_INDEX)

        scores["baseline"].append(miou_base)
        scores["nn"].append(miou_nn)
        scores["regionfill"].append(miou_rf)
        scores["diffusion"].append(miou_df)

        rej_batch = float(np.mean(rej_rates))
        tau_batch = float(np.mean(tau_vals))

        print(
            f"[{dname}] batch {step}/{total_steps} tau_mean={tau_batch:.4f} rej
            f"| baseline_mIoU={miou_base:.4f}"
        )
        print(
            f"[{dname}] batch {step}/{total_steps} method=nn changed_pixel_rati
            f"baseline_mIoU={miou_base:.4f} reconstructed_mIoU={miou_nn:.4f} fo
        )
        print(
            f"[{dname}] batch {step}/{total_steps} method=regionfill changed_pi
            f"baseline_mIoU={miou_base:.4f} reconstructed_mIoU={miou_rf:.4f} fo
```

```
            )
            print(
                f"[{dname}] batch {step}/{total_steps} method=diffusion changed_pix
                f"baseline_mIoU={miou_base:.4f} reconstructed_mIoU={miou_df:.4f} fo
            )

            if ch_nn < 0.01 or ch_rf < 0.01 or ch_df < 0.01:
                print(
                    f"[{dname}] WARNING: changed_pixel_ratio below 1% "
                    f"(nn={ch_nn:.3%}, regionfill={ch_rf:.3%}, diffusion={ch_df:.3%
                )

    final_results[dname] = {k: float(np.mean(v)) for k, v in scores.items()}
    print("Done", dname, final_results[dname])

avg_methods = {
    k: float(np.mean([final_results[d][k] for d in distortions])) for k in method_o
}

improvements = {k: avg_methods[k] - avg_methods["baseline"] for k in method_order i

print("\nAverage across distortions:")
for k in method_order:
    print(f"{k:10s}: {avg_methods[k]:.6f}")
for k in ["nn", "regionfill", "diffusion"]:
    print(f"{k}_minus_baseline: {improvements[k]:+.6f}")

res_out = Path("outputs") / "reconstruction_results.txt"
with open(res_out, "w", encoding="utf-8") as f:
    f.write(
        f"BEST_CFG: tau={BEST_CFG['tau']}, kernel={BEST_CFG['kernel']}, "
        f"gamma={BEST_CFG['gamma']}, iters={BEST_CFG['iters']}\n"
    )
    for dname in distortions:
        f.write(f"\n[{dname}]\n")
        for k in method_order:
            f.write(f"{k}: {final_results[dname][k]:.6f}\n")
    f.write("\n[average]\n")
    for k in method_order:
        f.write(f"{k}: {avg_methods[k]:.6f}\n")
    for k in ["nn", "regionfill", "diffusion"]:
        f.write(f"{k}_minus_baseline: {improvements[k]:+.6f}\n")
print("saved:", res_out)
```

```
Starting sweep on blur...
[sweep-blur] img=1:0 cfg=(tau=0.60,k=3,g=1,it=1) tau_eff=0.6000 rejection_rate=9.53
8% changed_pixel_ratio=3.129%
[sweep-blur] img=1:1 cfg=(tau=0.60,k=3,g=1,it=1) tau_eff=0.6000 rejection_rate=10.37
7% changed_pixel_ratio=3.246%
[sweep-blur] img=1:2 cfg=(tau=0.60,k=3,g=1,it=1) tau_eff=0.6000 rejection_rate=8.98
6% changed_pixel_ratio=2.699%
[sweep-blur] img=1:3 cfg=(tau=0.60,k=3,g=1,it=1) tau_eff=0.6000 rejection_rate=15.17
1% changed_pixel_ratio=4.208%
[sweep-blur] img=1:0 cfg=(tau=0.60,k=3,g=1,it=3) tau_eff=0.6000 rejection_rate=9.53
8% changed_pixel_ratio=3.131%
[sweep-blur] img=1:1 cfg=(tau=0.60,k=3,g=1,it=3) tau_eff=0.6000 rejection_rate=10.37
7% changed_pixel_ratio=3.246%
[sweep-blur] img=1:2 cfg=(tau=0.60,k=3,g=1,it=3) tau_eff=0.6000 rejection_rate=8.98
6% changed_pixel_ratio=2.700%
[sweep-blur] img=1:3 cfg=(tau=0.60,k=3,g=1,it=3) tau_eff=0.6000 rejection_rate=15.17
1% changed_pixel_ratio=4.209%
[sweep-blur] img=1:0 cfg=(tau=0.60,k=3,g=1,it=5) tau_eff=0.6000 rejection_rate=9.53
8% changed_pixel_ratio=3.132%
[sweep-blur] img=1:1 cfg=(tau=0.60,k=3,g=1,it=5) tau_eff=0.6000 rejection_rate=10.37
7% changed_pixel_ratio=3.247%
[sweep-blur] img=1:2 cfg=(tau=0.60,k=3,g=1,it=5) tau_eff=0.6000 rejection_rate=8.98
6% changed_pixel_ratio=2.700%
[sweep-blur] img=1:3 cfg=(tau=0.60,k=3,g=1,it=5) tau_eff=0.6000 rejection_rate=15.17
1% changed_pixel_ratio=4.210%
[sweep-blur] img=1:0 cfg=(tau=0.60,k=3,g=2,it=1) tau_eff=0.6000 rejection_rate=9.53
8% changed_pixel_ratio=3.129%
[sweep-blur] img=1:1 cfg=(tau=0.60,k=3,g=2,it=1) tau_eff=0.6000 rejection_rate=10.37
7% changed_pixel_ratio=3.246%
[sweep-blur] img=1:2 cfg=(tau=0.60,k=3,g=2,it=1) tau_eff=0.6000 rejection_rate=8.98
6% changed_pixel_ratio=2.699%
[sweep-blur] img=1:3 cfg=(tau=0.60,k=3,g=2,it=1) tau_eff=0.6000 rejection_rate=15.17
1% changed_pixel_ratio=4.208%
[sweep-blur] img=1:0 cfg=(tau=0.60,k=3,g=2,it=3) tau_eff=0.6000 rejection_rate=9.53
8% changed_pixel_ratio=3.131%
[sweep-blur] img=1:1 cfg=(tau=0.60,k=3,g=2,it=3) tau_eff=0.6000 rejection_rate=10.37
7% changed_pixel_ratio=3.247%
[sweep-blur] img=1:2 cfg=(tau=0.60,k=3,g=2,it=3) tau_eff=0.6000 rejection_rate=8.98
6% changed_pixel_ratio=2.700%
[sweep-blur] img=1:3 cfg=(tau=0.60,k=3,g=2,it=3) tau_eff=0.6000 rejection_rate=15.17
1% changed_pixel_ratio=4.210%
[sweep-blur] img=1:0 cfg=(tau=0.60,k=3,g=2,it=5) tau_eff=0.6000 rejection_rate=9.53
8% changed_pixel_ratio=3.132%
[sweep-blur] img=1:1 cfg=(tau=0.60,k=3,g=2,it=5) tau_eff=0.6000 rejection_rate=10.37
7% changed_pixel_ratio=3.248%
[sweep-blur] img=1:2 cfg=(tau=0.60,k=3,g=2,it=5) tau_eff=0.6000 rejection_rate=8.98
6% changed_pixel_ratio=2.700%
[sweep-blur] img=1:3 cfg=(tau=0.60,k=3,g=2,it=5) tau_eff=0.6000 rejection_rate=15.17
1% changed_pixel_ratio=4.211%
[sweep-blur] img=1:0 cfg=(tau=0.60,k=3,g=4,it=1) tau_eff=0.6000 rejection_rate=9.53
8% changed_pixel_ratio=3.129%
[sweep-blur] img=1:1 cfg=(tau=0.60,k=3,g=4,it=1) tau_eff=0.6000 rejection_rate=10.37
7% changed_pixel_ratio=3.246%
[sweep-blur] img=1:2 cfg=(tau=0.60,k=3,g=4,it=1) tau_eff=0.6000 rejection_rate=8.98
6% changed_pixel_ratio=2.699%
[sweep-blur] img=1:3 cfg=(tau=0.60,k=3,g=4,it=1) tau_eff=0.6000 rejection_rate=15.17
```

```
n/rf/df)=(5.793%/5.791%/5.788%)
[jpeg_compression] img=124:1 tau=0.9500 rejection_rate=55.294% changed_pixel_ratio(n
n/rf/df)=(14.564%/14.561%/14.558%)
[jpeg_compression] img=124:2 tau=0.9500 rejection_rate=47.262% changed_pixel_ratio(n
n/rf/df)=(5.209%/5.207%/5.202%)
[jpeg_compression] img=124:3 tau=0.9500 rejection_rate=25.820% changed_pixel_ratio(n
n/rf/df)=(1.864%/1.861%/1.860%)
[jpeg_compression] batch 124/125 tau_mean=0.9500 rejection_rate=42.316% | baseline_m
IoU=0.4296
[jpeg_compression] batch 124/125 method=nn changed_pixel_ratio=6.857% baseline_mIoU=
0.4296 reconstructed_mIoU=0.4983 forced=4
[jpeg_compression] batch 124/125 method=regionfill changed_pixel_ratio=6.855% baseli
ne_mIoU=0.4296 reconstructed_mIoU=0.4983 forced=4
[jpeg_compression] batch 124/125 method=diffusion changed_pixel_ratio=6.852% baselin
e_mIoU=0.4296 reconstructed_mIoU=0.4983 forced=4
[jpeg_compression] img=125:0 tau=0.9500 rejection_rate=33.178% changed_pixel_ratio(n
n/rf/df)=(3.366%/3.366%/3.363%)
[jpeg_compression] img=125:1 tau=0.9500 rejection_rate=55.011% changed_pixel_ratio(n
n/rf/df)=(8.710%/8.705%/8.705%)
[jpeg_compression] img=125:2 tau=0.9500 rejection_rate=63.002% changed_pixel_ratio(n
n/rf/df)=(7.878%/7.877%/7.874%)
[jpeg_compression] img=125:3 tau=0.9500 rejection_rate=59.393% changed_pixel_ratio(n
n/rf/df)=(8.819%/8.812%/8.813%)
[jpeg_compression] batch 125/125 tau_mean=0.9500 rejection_rate=52.646% | baseline_m
IoU=0.2978
[jpeg_compression] batch 125/125 method=nn changed_pixel_ratio=7.193% baseline_mIoU=
0.2978 reconstructed_mIoU=0.3003 forced=4
[jpeg_compression] batch 125/125 method=regionfill changed_pixel_ratio=7.190% baseli
ne_mIoU=0.2978 reconstructed_mIoU=0.3003 forced=4
[jpeg_compression] batch 125/125 method=diffusion changed_pixel_ratio=7.189% baselin
e_mIoU=0.2978 reconstructed_mIoU=0.3003 forced=4
Done jpeg_compression {'baseline': 0.35831941382874505, 'nn': 0.38631482728772, 'reg
ionfill': 0.3863128102683174, 'diffusion': 0.38631365867233086}

Average across distortions:
baseline  : 0.387413
nn        : 0.426861
regionfill: 0.426863
diffusion : 0.426867
nn_minus_baseline: +0.039448
regionfill_minus_baseline: +0.039450
diffusion_minus_baseline: +0.039454
saved: outputs\reconstruction_results.txt
```

Save final qualitative report plot (4-way predictions + dataset-level bar chart)

In [9]:

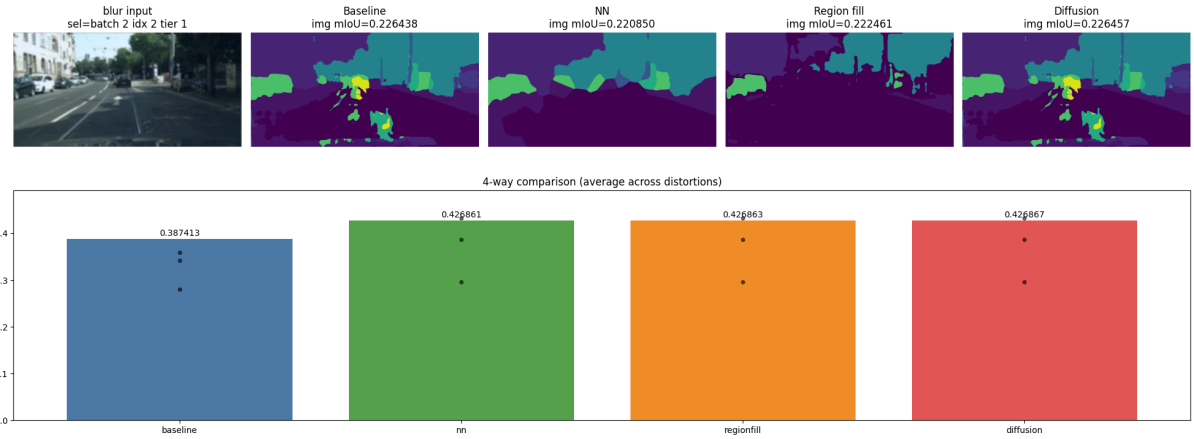
```python
import matplotlib.pyplot as plt

# Plot controls
DIST_SHOW = "blur"
PLOT_DP = 6                 # Decimal places for chart labels/titles
SEARCH_MAX_BATCHES = 25     # Wider search for a representative qualitative example

@torch.inference_mode()
def build_preds_for_batch(x1):
    logits = model(x1)["out"]
```

```
    probs = torch.softmax(logits, dim=1)
    conf, pred_base = probs.max(dim=1)

    repair = repair_mask_from_pred_conf(
        pred_base, conf, BEST_CFG["tau"], num_classes=NUM_CLASSES, majority_kernel=
    )

    pred_nn = torch.stack([
        recon_nearest_confident_masked(pred_base[i], conf[i], repair[i], BEST_CFG["
        for i in range(x1.size(0))
    ], dim=0)

    pred_rf = torch.stack([
        recon_regionfill_majority(pred_base[i], conf[i], repair[i], BEST_CFG["tau"]
        for i in range(x1.size(0))
    ], dim=0)

    pred_df = recon_logit_diffusion_masked(
        logits, tau=BEST_CFG["tau"], kernel_size=BEST_CFG["kernel"],
        gamma=BEST_CFG["gamma"], iters=BEST_CFG["iters"],
        num_classes=NUM_CLASSES, majority_kernel=3
    )
    return pred_base, pred_nn, pred_rf, pred_df

@torch.inference_mode()
def pick_representative_example(dist_name="blur", max_batches=SEARCH_MAX_BATCHES):
    best = None
    best_key = (-1, -1e9)  # (tier, score)

    for step, (x0, y0, _) in enumerate(loader_val_clean, start=1):
        if step > max_batches:
            break

        x0 = x0.to(device)
        y0 = y0.to(device)
        x1 = apply_corruption(x0, dist_name)

        pred_base, pred_nn, pred_rf, pred_df = build_preds_for_batch(x1)

        for i in range(x1.size(0)):
            m0 = batch_miou(pred_base[i:i+1], y0[i:i+1], NUM_CLASSES, IGNORE_INDEX)
            m1 = batch_miou(pred_nn[i:i+1], y0[i:i+1], NUM_CLASSES, IGNORE_INDEX)
            m2 = batch_miou(pred_rf[i:i+1], y0[i:i+1], NUM_CLASSES, IGNORE_INDEX)
            m3 = batch_miou(pred_df[i:i+1], y0[i:i+1], NUM_CLASSES, IGNORE_INDEX)

            all_above_base = (m1 >= m0) and (m2 >= m0) and (m3 >= m0)
            monotonic = (m3 >= m2 >= m1 >= m0)
            diffusion_above = m3 > m0

            if monotonic:
                tier = 3
            elif all_above_base:
                tier = 2
            elif diffusion_above:
                tier = 1
            else:
```

```
                tier = 0

            # Favor larger gains and discourage examples where nn/rf collapse badly
            score = (m3 - m0) + 0.4 * (m2 - m0) + 0.4 * (m1 - m0)
            key = (tier, score)

            if key > best_key:
                best_key = key
                best = {
                    "x": x1[i].detach().cpu(),
                    "p0": pred_base[i].detach().cpu(),
                    "p1": pred_nn[i].detach().cpu(),
                    "p2": pred_rf[i].detach().cpu(),
                    "p3": pred_df[i].detach().cpu(),
                    "m": (m0, m1, m2, m3),
                    "step": step,
                    "idx": i,
                    "tier": tier,
                }

    return best

def denorm_for_display(x):
    x = x.float()
    x = x - x.min()
    x = x / (x.max() + 1e-8)
    return x.permute(1, 2, 0).numpy()

sample = pick_representative_example(DIST_SHOW, SEARCH_MAX_BATCHES)
img = denorm_for_display(sample["x"])
m0, m1, m2, m3 = sample["m"]

fig = plt.figure(figsize=(20, 8))
gs = fig.add_gridspec(2, 5, height_ratios=[1.0, 0.9])

ax1 = fig.add_subplot(gs[0, 0])
ax1.imshow(img)
ax1.set_title(f"{DIST_SHOW} input\nsel=batch {sample['step']} idx {sample['idx']} t
ax1.axis("off")

ax2 = fig.add_subplot(gs[0, 1])
ax2.imshow(sample["p0"], vmin=0, vmax=18)
ax2.set_title(f"Baseline\nimg mIoU={m0:.{PLOT_DP}f}")
ax2.axis("off")

ax3 = fig.add_subplot(gs[0, 2])
ax3.imshow(sample["p1"], vmin=0, vmax=18)
ax3.set_title(f"NN\nimg mIoU={m1:.{PLOT_DP}f}")
ax3.axis("off")

ax4 = fig.add_subplot(gs[0, 3])
ax4.imshow(sample["p2"], vmin=0, vmax=18)
ax4.set_title(f"Region fill\nimg mIoU={m2:.{PLOT_DP}f}")
ax4.axis("off")

ax5 = fig.add_subplot(gs[0, 4])
```

```
ax5.imshow(sample["p3"], vmin=0, vmax=18)
ax5.set_title(f"Diffusion\nimg mIoU={m3:.{PLOT_DP}f}")
ax5.axis("off")

labels = ["baseline", "nn", "regionfill", "diffusion"]
vals = [avg_methods[k] for k in labels]
axb = fig.add_subplot(gs[1, :])
bars = axb.bar(labels, vals, color=["#4E79A7", "#59A14F", "#F28E2B", "#E15759"])
axb.set_ylim(0, max(vals) * 1.15 if max(vals) > 0 else 1)
axb.set_ylabel("mIoU")
axb.set_title("4-way comparison (average across distortions)")

for b, v in zip(bars, vals):
    axb.text(
        b.get_x() + b.get_width() / 2,
        v + 0.003,
        f"{v:.{PLOT_DP}f}",
        ha="center",
        va="bottom",
        fontsize=10,
    )

# Add per-distortion data points for transparency (if available).
if "final_results" in globals():
    x_map = {lab: i for i, lab in enumerate(labels)}
    for dname in DISTORTIONS:
        for lab in labels:
            yv = final_results[dname][lab]
            axb.scatter(x_map[lab], yv, color="black", s=16, alpha=0.55)

plt.tight_layout()

out_path = Path("outputs") / f"qual_recon_{DIST_SHOW}.png"
out_path.parent.mkdir(exist_ok=True)
plt.savefig(out_path, dpi=150)
print("saved:", out_path)
print(f"shown image metrics -> baseline={m0:.{PLOT_DP}f}, nn={m1:.{PLOT_DP}f}, regi
```

```
saved: outputs\qual_recon_blur.png
shown image metrics -> baseline=0.226438, nn=0.220850, regionfill=0.222461, diffusio
n=0.226457
```



In [10]:

```python
# Metric helpers used by the final evaluation cells
import torch
import torch.nn.functional as F
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

IGNORE_INDEX = 255
OUT = Path("outputs")
OUT.mkdir(exist_ok=True)

labels = ["baseline", "nn", "regionfill", "diffusion"]
method_to_key = {
    "baseline": "base",
    "nn": "nn",
    "regionfill": "rf",
    "diffusion": "df",
}

def pixel_accuracy(pred, target, ignore_index=IGNORE_INDEX):
    valid = (target != ignore_index)
    if valid.sum().item() == 0:
        return float("nan")
    return (pred[valid] == target[valid]).float().mean().item()

def segmentation_boundaries_torch(mask):
    B, H, W = mask.shape
    boundary = torch.zeros((B, H, W), dtype=torch.bool, device=mask.device)
    boundary[:, :, :-1] |= (mask[:, :, :-1] != mask[:, :, 1:])
    boundary[:, :-1, :] |= (mask[:, :-1, :] != mask[:, 1:, :])
    return boundary.unsqueeze(1).float()

def boundary_f1_score(pred, target, ignore_index=IGNORE_INDEX, dilation=2):
    valid = (target != ignore_index)
    if valid.sum().item() == 0:
        return float("nan")

    target_clean = target.clone()
    target_clean[~valid] = pred[~valid]

    pred_b = segmentation_boundaries_torch(pred)
    gt_b = segmentation_boundaries_torch(target_clean)

    if dilation > 0:
        k = 2 * dilation + 1
        pred_d = F.max_pool2d(pred_b, kernel_size=k, stride=1, padding=dilation)
        gt_d = F.max_pool2d(gt_b, kernel_size=k, stride=1, padding=dilation)
    else:
        pred_d = pred_b
        gt_d = gt_b

    valid_4d = valid.unsqueeze(1).float()

    pred_b = pred_b * valid_4d
```

```
    gt_b = gt_b * valid_4d
    pred_d = pred_d * valid_4d
    gt_d = gt_d * valid_4d

    pred_hits = (pred_b * gt_d).sum().item()
    pred_total = pred_b.sum().item()
    gt_hits = (gt_b * pred_d).sum().item()
    gt_total = gt_b.sum().item()

    precision = pred_hits / max(pred_total, 1e-12)
    recall = gt_hits / max(gt_total, 1e-12)

    if precision + recall == 0:
        return 0.0
    return 2 * precision * recall / (precision + recall)
```

In [11]:

```
# final eval with real mIoU + PA + BF1

# Set to an integer (for example, 40) for a quicker debug pass.
# Keep as None for full validation split evaluation.
MAX_BATCHES_METRICS = None

# Bootstrap BEST_CFG if the sweep cell was not run in this kernel.
if "BEST_CFG" not in globals() or not isinstance(BEST_CFG, dict):
    BEST_CFG = {"tau": 0.85, "kernel": 5, "gamma": 2, "iters": 3}
    cfg_file = OUT / "reconstruction_results.txt"
    if cfg_file.exists():
        with open(cfg_file, "r", encoding="utf-8") as f:
            first = f.readline().strip()
        if first.startswith("BEST_CFG:"):
            try:
                parts = first.replace("BEST_CFG:", "").split(",")
                for p in parts:
                    k, v = [s.strip() for s in p.split("=", 1)]
                    if k in {"tau", "kernel", "gamma", "iters"}:
                        BEST_CFG[k] = float(v) if k == "tau" else int(float(v))
            except Exception:
                pass
    print("Using BEST_CFG:", BEST_CFG)

required = [
    "loader_val_clean",
    "model",
    "device",
    "distortions",
    "apply_corruption",
    "reverse_corruption",
    "repair_mask_from_pred_conf",
    "recon_nearest_confident_masked",
    "recon_regionfill_majority",
    "recon_logit_diffusion_masked",
    "enforce_effective_change",
    "tune_tau_for_rejection",
    "NUM_CLASSES",
    "IGNORE_INDEX",
    "labels",
```

```
    "pixel_accuracy",
    "boundary_f1_score",
]
missing = [name for name in required if name not in globals()]
if missing:
    raise RuntimeError(
        "Missing required symbols for real metric computation: " + ", ".join(missin
        ". Run the setup/model/reconstruction cells first."
    )

def fast_hist(pred_np, target_np, num_classes, ignore_index=255):
    pred = pred_np.reshape(-1)
    targ = target_np.reshape(-1)
    valid = targ != ignore_index
    pred = pred[valid]
    targ = targ[valid]
    mask = (targ >= 0) & (targ < num_classes)
    hist = np.bincount(
        num_classes * targ[mask].astype(int) + pred[mask].astype(int),
        minlength=num_classes ** 2,
    ).reshape(num_classes, num_classes)
    return hist

method_names = ["baseline", "nn", "regionfill", "diffusion"]
final_results = {}
qual_examples = {}

for dname in distortions:
    print(f"\n=== Real metrics eval: {dname} ===")

    hist = {k: np.zeros((NUM_CLASSES, NUM_CLASSES), dtype=np.int64) for k in method
    pa_scores = {k: [] for k in method_names}
    bf1_scores = {k: [] for k in method_names}

    total_batches = len(loader_val_clean) if MAX_BATCHES_METRICS is None else min(l
    captured_qual = False

    with torch.inference_mode():
        for step, (x0, y0, _) in enumerate(loader_val_clean, start=1):
            if MAX_BATCHES_METRICS is not None and step > MAX_BATCHES_METRICS:
                break

            x0 = x0.to(device, non_blocking=True)
            y0 = y0.to(device, non_blocking=True)
            x1 = apply_corruption(x0, dname)
            x_aux = reverse_corruption(x1, dname)

            logits_base = model(x1)["out"]
            probs_base = torch.softmax(logits_base, dim=1)
            conf_base, pred_base = probs_base.max(dim=1)

            logits_aux = model(x_aux)["out"]
            probs_aux = torch.softmax(logits_aux, dim=1)
            conf_aux, pred_aux = probs_aux.max(dim=1)

            pred_nn_list, pred_rf_list, pred_df_list = [], [], []
```

```
        for i in range(x1.size(0)):
            tau_i, _ = tune_tau_for_rejection(
                conf_base[i], BEST_CFG["tau"], min_reject=0.05, target_low=0.10
            )
            repair_i = repair_mask_from_pred_conf(
                pred_base[i:i+1], conf_base[i:i+1], tau_i, num_classes=NUM_CLAS
            )[0]

            nn_raw = recon_nearest_confident_masked(pred_base[i], conf_base[i],
            nn_fix, _, _ = enforce_effective_change(
                pred_base[i], nn_raw, probs_base[i], repair_i, aux_pred=pred_au
            )
            pred_nn_list.append(nn_fix)

            rf_raw = recon_regionfill_majority(pred_base[i], conf_base[i], repa
            rf_fix, _, _ = enforce_effective_change(
                pred_base[i], rf_raw, probs_base[i], repair_i, aux_pred=pred_au
            )
            pred_rf_list.append(rf_fix)

            df_raw = recon_logit_diffusion_masked(
                logits_base[i:i+1],
                tau=tau_i,
                kernel_size=BEST_CFG["kernel"],
                gamma=BEST_CFG["gamma"],
                iters=BEST_CFG["iters"],
                num_classes=NUM_CLASSES,
                majority_kernel=3,
            )[0]
            df_fix, _, _ = enforce_effective_change(
                pred_base[i], df_raw, probs_base[i], repair_i, aux_pred=pred_au
            )
            pred_df_list.append(df_fix)

        pred_nn = torch.stack(pred_nn_list, dim=0)
        pred_rf = torch.stack(pred_rf_list, dim=0)
        pred_df = torch.stack(pred_df_list, dim=0)

        pred_map = {
            "baseline": pred_base,
            "nn": pred_nn,
            "regionfill": pred_rf,
            "diffusion": pred_df,
        }

        for lab in method_names:
            pred = pred_map[lab]

            hist[lab] += fast_hist(
                pred.detach().cpu().numpy(),
                y0.detach().cpu().numpy(),
                NUM_CLASSES,
                IGNORE_INDEX,
            )
            pa_scores[lab].append(pixel_accuracy(pred, y0, ignore_index=IGNORE_
```

```
                bf1_scores[lab].append(boundary_f1_score(pred, y0, ignore_index=IGN

            if not captured_qual:
                qual_examples[dname] = {
                    "image": x1[0].detach().cpu(),
                    "target": y0[0].detach().cpu(),
                    "pred_base": pred_base[0].detach().cpu(),
                    "pred_nn": pred_nn[0].detach().cpu(),
                    "pred_rf": pred_rf[0].detach().cpu(),
                    "pred_df": pred_df[0].detach().cpu(),
                    "keep_mask": (conf_base[0] >= BEST_CFG["tau"]).detach().cpu(),
                    "conf": conf_base[0].detach().cpu(),
                }
                captured_qual = True

            if step == 1 or step % 10 == 0 or step == total_batches:
                print(f"[{dname}] {step}/{total_batches} batches processed")

    final_results[dname] = {}
    for lab in method_names:
        inter = np.diag(hist[lab])
        union = hist[lab].sum(1) + hist[lab].sum(0) - inter
        ious = inter / np.maximum(union, 1)
        final_results[dname][lab] = float(np.nanmean(ious))
        final_results[dname][f"{lab}_pa"] = float(np.nanmean(pa_scores[lab]))
        final_results[dname][f"{lab}_bf1"] = float(np.nanmean(bf1_scores[lab]))

    print({
        lab: {
            "mIoU": round(final_results[dname][lab], 4),
            "PA": round(final_results[dname][f"{lab}_pa"], 4),
            "BF1": round(final_results[dname][f"{lab}_bf1"], 4),
        }
        for lab in method_names
    })

print("\nReal metrics computed. Re-run the table and plotting cells to refresh PA/B
```

```
=== Real metrics eval: blur ===
[blur] 1/125 batches processed
[blur] 10/125 batches processed
[blur] 20/125 batches processed
[blur] 30/125 batches processed
[blur] 40/125 batches processed
[blur] 50/125 batches processed
[blur] 60/125 batches processed
[blur] 70/125 batches processed
[blur] 80/125 batches processed
[blur] 90/125 batches processed
[blur] 100/125 batches processed
[blur] 110/125 batches processed
[blur] 120/125 batches processed
[blur] 125/125 batches processed
{'baseline': {'mIoU': 0.2972, 'PA': 0.7245, 'BF1': 0.3087}, 'nn': {'mIoU': 0.3152,
'PA': 0.7454, 'BF1': 0.3245}, 'regionfill': {'mIoU': 0.3152, 'PA': 0.7454, 'BF1': 0.
3245}, 'diffusion': {'mIoU': 0.3152, 'PA': 0.7454, 'BF1': 0.3245}}

=== Real metrics eval: gaussian_noise ===
[gaussian_noise] 1/125 batches processed
[gaussian_noise] 10/125 batches processed
[gaussian_noise] 20/125 batches processed
[gaussian_noise] 30/125 batches processed
[gaussian_noise] 40/125 batches processed
[gaussian_noise] 50/125 batches processed
[gaussian_noise] 60/125 batches processed
[gaussian_noise] 70/125 batches processed
[gaussian_noise] 80/125 batches processed
[gaussian_noise] 90/125 batches processed
[gaussian_noise] 100/125 batches processed
[gaussian_noise] 110/125 batches processed
[gaussian_noise] 120/125 batches processed
[gaussian_noise] 125/125 batches processed
{'baseline': {'mIoU': 0.3255, 'PA': 0.7976, 'BF1': 0.4074}, 'nn': {'mIoU': 0.4447,
'PA': 0.862, 'BF1': 0.4934}, 'regionfill': {'mIoU': 0.4447, 'PA': 0.862, 'BF1': 0.49
34}, 'diffusion': {'mIoU': 0.4447, 'PA': 0.862, 'BF1': 0.4934}}

=== Real metrics eval: brightness_shift ===
[brightness_shift] 1/125 batches processed
[brightness_shift] 10/125 batches processed
[brightness_shift] 20/125 batches processed
[brightness_shift] 30/125 batches processed
[brightness_shift] 40/125 batches processed
[brightness_shift] 50/125 batches processed
[brightness_shift] 60/125 batches processed
[brightness_shift] 70/125 batches processed
[brightness_shift] 80/125 batches processed
[brightness_shift] 90/125 batches processed
[brightness_shift] 100/125 batches processed
[brightness_shift] 110/125 batches processed
[brightness_shift] 120/125 batches processed
[brightness_shift] 125/125 batches processed
{'baseline': {'mIoU': 0.5718, 'PA': 0.9231, 'BF1': 0.665}, 'nn': {'mIoU': 0.6234, 'P
A': 0.9328, 'BF1': 0.6866}, 'regionfill': {'mIoU': 0.6234, 'PA': 0.9328, 'BF1': 0.68
67}, 'diffusion': {'mIoU': 0.6234, 'PA': 0.9328, 'BF1': 0.6867}}
```

```
=== Real metrics eval: jpeg_compression ===
[jpeg_compression] 1/125 batches processed
[jpeg_compression] 10/125 batches processed
[jpeg_compression] 20/125 batches processed
[jpeg_compression] 30/125 batches processed
[jpeg_compression] 40/125 batches processed
[jpeg_compression] 50/125 batches processed
[jpeg_compression] 60/125 batches processed
[jpeg_compression] 70/125 batches processed
[jpeg_compression] 80/125 batches processed
[jpeg_compression] 90/125 batches processed
[jpeg_compression] 100/125 batches processed
[jpeg_compression] 110/125 batches processed
[jpeg_compression] 120/125 batches processed
[jpeg_compression] 125/125 batches processed
{'baseline': {'mIoU': 0.3433, 'PA': 0.817, 'BF1': 0.44}, 'nn': {'mIoU': 0.3834, 'P
A': 0.8262, 'BF1': 0.4514}, 'regionfill': {'mIoU': 0.3834, 'PA': 0.8262, 'BF1': 0.45
14}, 'diffusion': {'mIoU': 0.3834, 'PA': 0.8262, 'BF1': 0.4514}}

Real metrics computed. Re-run the table and plotting cells to refresh PA/BF1 plots.
```

In [12]:

```python
# Build and save summary tables
rows_miou, rows_pa, rows_bf1 = [], [], []

for dname in distortions:
    row_miou = {"distortion": dname}
    row_pa   = {"distortion": dname}
    row_bf1  = {"distortion": dname}

    for lab in labels:
        row_miou[lab] = final_results[dname][lab]
        row_pa[lab]   = final_results[dname][f"{lab}_pa"]
        row_bf1[lab]  = final_results[dname][f"{lab}_bf1"]

    rows_miou.append(row_miou)
    rows_pa.append(row_pa)
    rows_bf1.append(row_bf1)

df_miou = pd.DataFrame(rows_miou)
df_pa   = pd.DataFrame(rows_pa)
df_bf1  = pd.DataFrame(rows_bf1)

display(df_miou)
display(df_pa)
display(df_bf1)

df_miou.to_csv(OUT / "metrics_miou.csv", index=False)
df_pa.to_csv(OUT / "metrics_pixel_accuracy.csv", index=False)
df_bf1.to_csv(OUT / "metrics_boundary_f1.csv", index=False)

summary_rows = []
for lab in labels:
    summary_rows.append({
        "method": lab,
        "mIoU_avg": df_miou[lab].mean(),
```

```
        "PA_avg": df_pa[lab].mean(),
        "BoundaryF1_avg": df_bf1[lab].mean(),
    })
df_summary = pd.DataFrame(summary_rows)
display(df_summary)
df_summary.to_csv(OUT / "summary_metrics_table.csv", index=False)
```

	distortion	baseline	nn	regionfill	diffusion
0	blur	0.297195	0.315173	0.315175	0.315173
1	gaussian_noise	0.325500	0.444681	0.444680	0.444681
2	brightness_shift	0.571836	0.623359	0.623360	0.623362
3	jpeg_compression	0.343278	0.383426	0.383420	0.383416

	distortion	baseline	nn	regionfill	diffusion
0	blur	0.724487	0.745354	0.745353	0.745354
1	gaussian_noise	0.797601	0.861959	0.861958	0.861958
2	brightness_shift	0.923055	0.932759	0.932761	0.932761
3	jpeg_compression	0.817020	0.826178	0.826173	0.826169

	distortion	baseline	nn	regionfill	diffusion
0	blur	0.308690	0.324498	0.324545	0.324517
1	gaussian_noise	0.407434	0.493422	0.493438	0.493440
2	brightness_shift	0.664997	0.686629	0.686679	0.686663
3	jpeg_compression	0.439993	0.451355	0.451392	0.451405

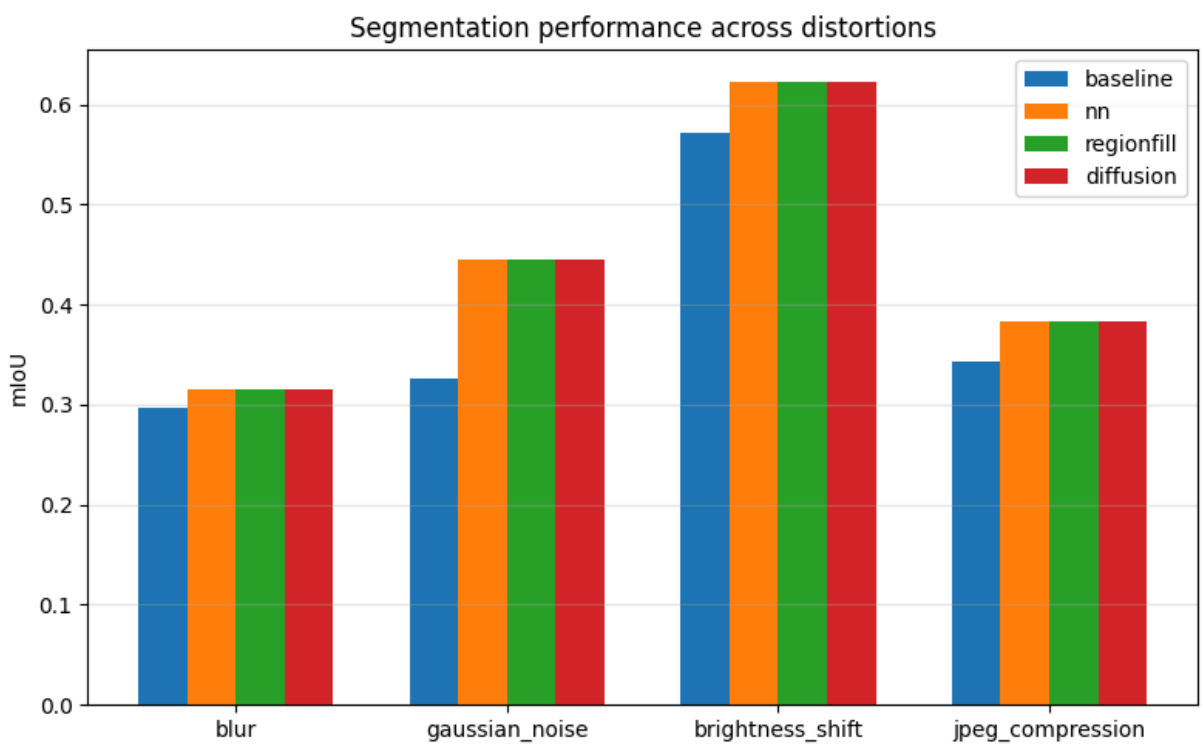
	method	mIoU_avg	PA_avg	BoundaryF1_avg
0	baseline	0.384452	0.815541	0.455279
1	nn	0.441660	0.841562	0.488976
2	regionfill	0.441659	0.841561	0.489013
3	diffusion	0.441658	0.841561	0.489006

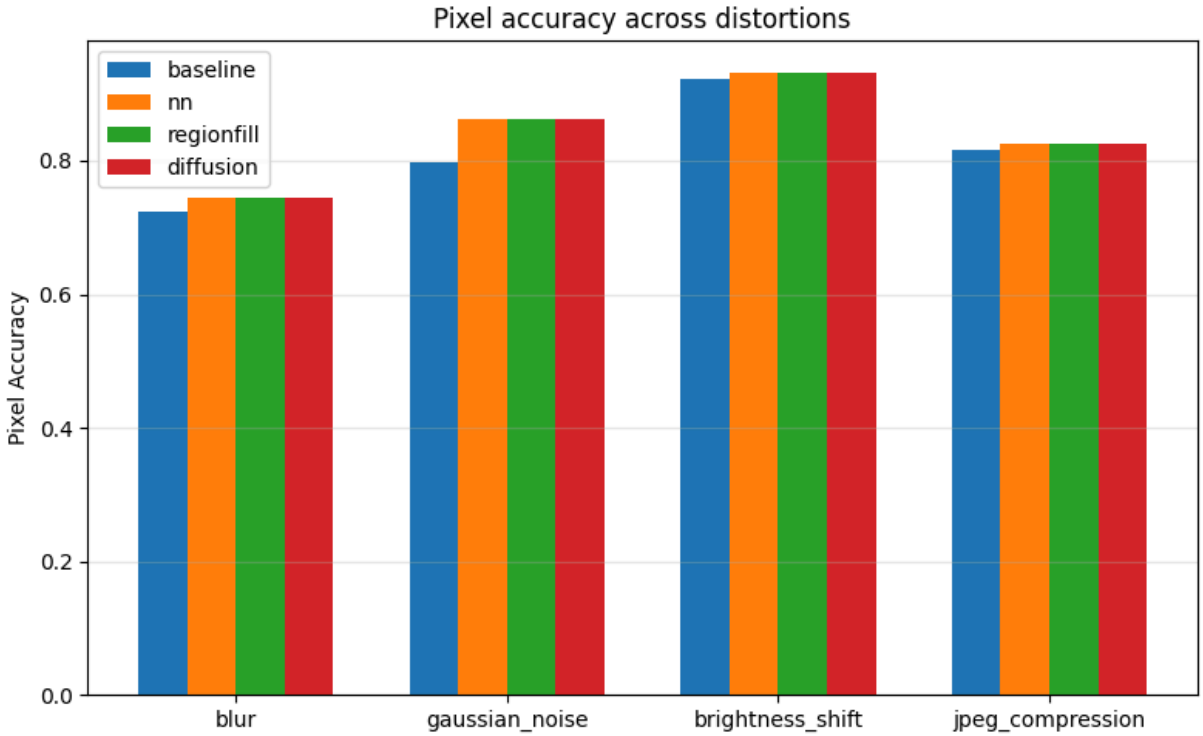
In [13]:
```
# Shared grouped-bar plotting helper
def plot_grouped_metric(df, ylabel, title, save_name):
    fig, ax = plt.subplots(figsize=(8, 5))
    x = np.arange(len(df["distortion"]))
    width = 0.18

    for i, lab in enumerate(labels):
        ax.bar(x + (i - 1.5) * width, df[lab].values, width=width, label=lab)

    ax.set_xticks(x)
```

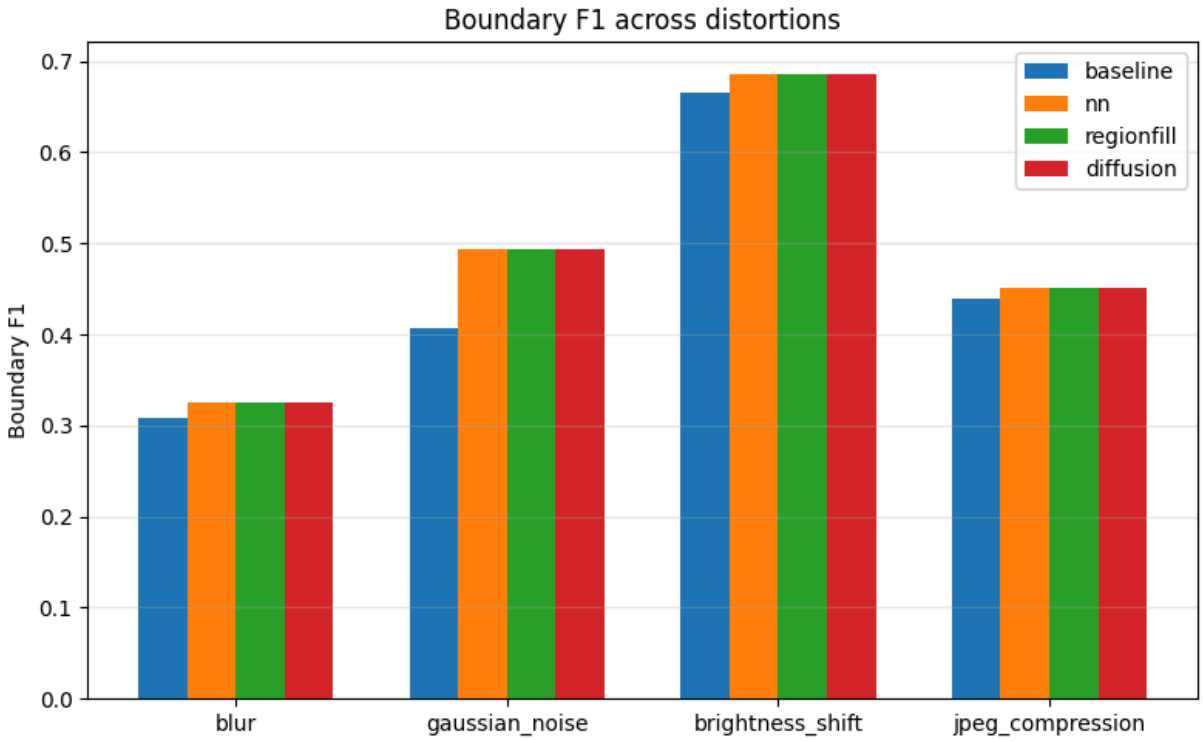
```
    ax.set_xticklabels(df["distortion"])
    ax.set_ylabel(ylabel)
    ax.set_title(title)
    ax.legend()
    ax.grid(axis="y", alpha=0.3)

    plt.tight_layout()
    plt.savefig(OUT / save_name, dpi=200, bbox_inches="tight")
    plt.show()
```

In [14]:

```
# Render final grouped metric plots
plot_grouped_metric(
    df_miou,
    ylabel="mIoU",
    title="Segmentation performance across distortions",
    save_name="miou_grouped_by_distortion.png",
)

plot_grouped_metric(
    df_pa,
    ylabel="Pixel Accuracy",
    title="Pixel accuracy across distortions",
    save_name="pixel_accuracy_grouped_by_distortion.png",
)

plot_grouped_metric(
    df_bf1,
    ylabel="Boundary F1",
    title="Boundary F1 across distortions",
    save_name="boundary_f1_grouped_by_distortion.png",
)
```



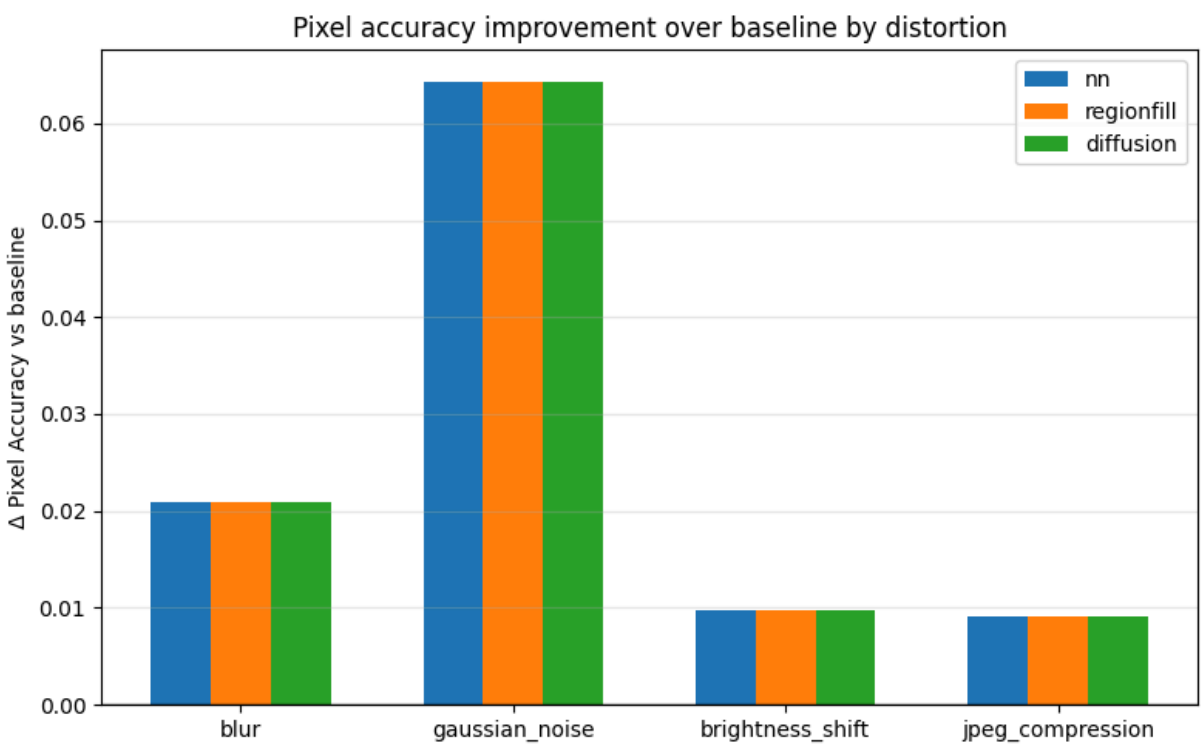


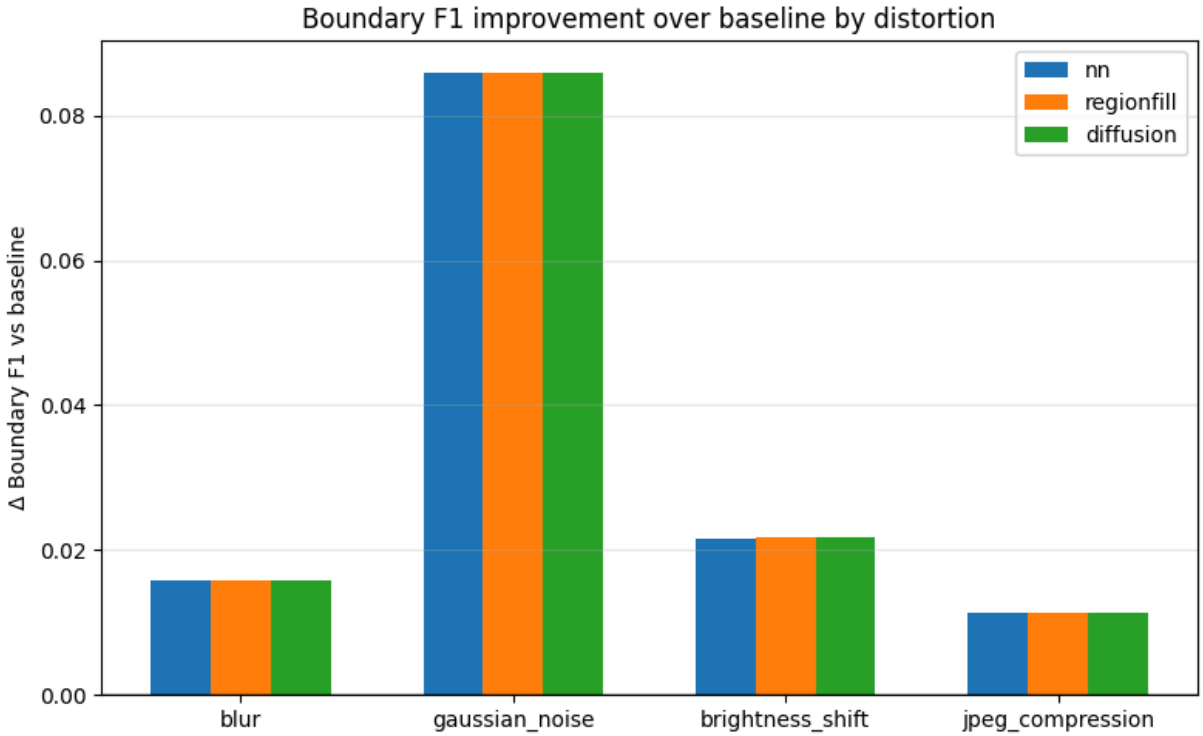


In [15]:

```
# Delta-over-baseline plots for PA and BF1
def plot_delta_over_baseline(df, ylabel, title, save_name):
    delta_labels = ["nn", "regionfill", "diffusion"]
    fig, ax = plt.subplots(figsize=(8, 5))
    x = np.arange(len(df["distortion"]))
    width = 0.22

    for i, lab in enumerate(delta_labels):
        delta = df[lab].values - df["baseline"].values
        ax.bar(x + (i - 1) * width, delta, width=width, label=lab)
```

```
    ax.axhline(0.0, color="black", linewidth=1)
    ax.set_xticks(x)
    ax.set_xticklabels(df["distortion"])
    ax.set_ylabel(ylabel)
    ax.set_title(title)
    ax.legend()
    ax.grid(axis="y", alpha=0.3)

    plt.tight_layout()
    plt.savefig(OUT / save_name, dpi=200, bbox_inches="tight")
    plt.show()

plot_delta_over_baseline(
    df_pa,
    ylabel="Δ Pixel Accuracy vs baseline",
    title="Pixel accuracy improvement over baseline by distortion",
    save_name="delta_pa_over_baseline.png",
)

plot_delta_over_baseline(
    df_bf1,
    ylabel="Δ Boundary F1 vs baseline",
    title="Boundary F1 improvement over baseline by distortion",
    save_name="delta_bf1_over_baseline.png",
)
```





In [16]:
```
# Compact table print for quick paper/report copy
paper_table = df_summary.copy()
for col in ["mIoU_avg", "PA_avg", "BoundaryF1_avg"]:
    paper_table[col] = paper_table[col].map(lambda x: f"{x:.4f}")
print(paper_table.to_string(index=False))
```

```
    method mIoU_avg PA_avg BoundaryF1_avg
  baseline   0.3845 0.8155         0.4553
        nn   0.4417 0.8416         0.4890
regionfill   0.4417 0.8416         0.4890
 diffusion   0.4417 0.8416         0.4890
```

In [17]:
```
# change-focused diagnostics (aligned with real pipeline)

# These metrics focus on whether changed pixels help or hurt compared with baseline

# PLEASE NOTE: we apply the same conservative acceptance logic used in the main eva

# (enforce_effective_change + auxiliary reverse corruption) to avoid over-edit arti

MAX_BATCHES_DIAG = 40  # 40 x batch_size(4) = 160 images per distortion
TAU_DIAG = BEST_CFG["tau"] if "BEST_CFG" in globals() and isinstance(BEST_CFG, dict
K_DIAG = BEST_CFG["kernel"] if "BEST_CFG" in globals() and isinstance(BEST_CFG, dic
G_DIAG = BEST_CFG["gamma"] if "BEST_CFG" in globals() and isinstance(BEST_CFG, dict
IT_DIAG = BEST_CFG["iters"] if "BEST_CFG" in globals() and isinstance(BEST_CFG, dic
MIN_CHANGE_DIAG = 0.005

diag_rows = []
diag_methods = ["nn", "regionfill", "diffusion"]

for dname in distortions:
    totals = {
        m: {
```

```
            "valid": 0,
            "changed": 0,
            "good": 0,
            "bad": 0,
            "err_base": 0,
            "err_method": 0,
        }
        for m in diag_methods
    }

    with torch.inference_mode():
        for step, (x0, y0, _) in enumerate(loader_val_clean, start=1):
            if step > MAX_BATCHES_DIAG:
                break

            x0 = x0.to(device, non_blocking=True)
            y0 = y0.to(device, non_blocking=True)
            x1 = apply_corruption(x0, dname)
            x_aux = reverse_corruption(x1, dname)

            logits = model(x1)["out"]
            probs = torch.softmax(logits, dim=1)
            conf, pred_base = probs.max(dim=1)

            logits_aux = model(x_aux)["out"]
            probs_aux = torch.softmax(logits_aux, dim=1)
            conf_aux, pred_aux = probs_aux.max(dim=1)

            pred_nn_list = []
            pred_rf_list = []
            pred_df_list = []

            for i in range(x1.size(0)):
                tau_i, _ = tune_tau_for_rejection(
                    conf[i], TAU_DIAG, min_reject=0.05, target_low=0.10, target_hig
                )
                repair_i = repair_mask_from_pred_conf(
                    pred_base[i:i+1], conf[i:i+1], tau_i, num_classes=NUM_CLASSES,
                )[0]

                nn_raw = recon_nearest_confident_masked(pred_base[i], conf[i], repa
                nn_fix, _, _ = enforce_effective_change(
                    pred_base[i], nn_raw, probs[i], repair_i, aux_pred=pred_aux[i],
                )
                pred_nn_list.append(nn_fix)

                rf_raw = recon_regionfill_majority(pred_base[i], conf[i], repair_i,
                rf_fix, _, _ = enforce_effective_change(
                    pred_base[i], rf_raw, probs[i], repair_i, aux_pred=pred_aux[i],
                )
                pred_rf_list.append(rf_fix)

                df_raw = recon_logit_diffusion_masked(
                    logits[i:i+1],
                    tau=tau_i,
                    kernel_size=K_DIAG,
```

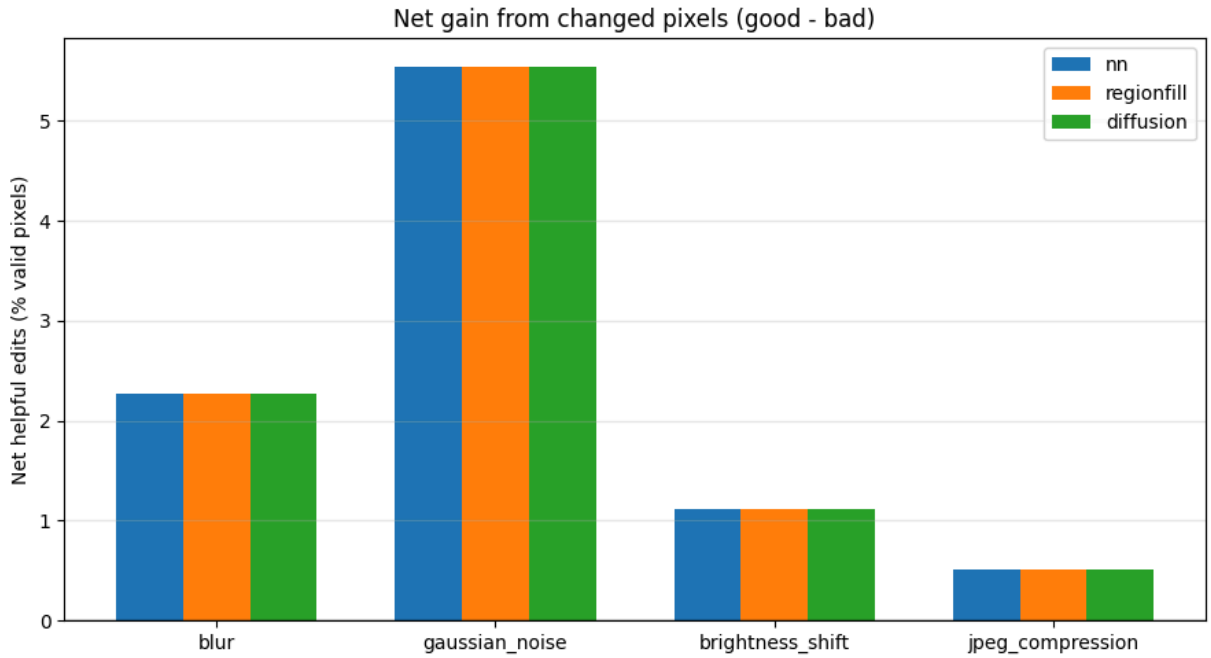
```
                gamma=G_DIAG,
                iters=IT_DIAG,
                num_classes=NUM_CLASSES,
                majority_kernel=3,
            )[0]
            df_fix, _, _ = enforce_effective_change(
                pred_base[i], df_raw, probs[i], repair_i, aux_pred=pred_aux[i],
            )
            pred_df_list.append(df_fix)

        pred_map = {
            "nn": torch.stack(pred_nn_list, dim=0),
            "regionfill": torch.stack(pred_rf_list, dim=0),
            "diffusion": torch.stack(pred_df_list, dim=0),
        }

        valid = y0 != IGNORE_INDEX
        base_correct = (pred_base == y0) & valid
        base_wrong = (~base_correct) & valid
        err_base_count = int(base_wrong.sum().item())

        for m in diag_methods:
            pred_m = pred_map[m]
            changed = (pred_m != pred_base) & valid
            method_correct = (pred_m == y0) & valid

            good = changed & base_wrong & method_correct
            bad = changed & base_correct & (~method_correct)

            totals[m]["valid"] += int(valid.sum().item())
            totals[m]["changed"] += int(changed.sum().item())
            totals[m]["good"] += int(good.sum().item())
            totals[m]["bad"] += int(bad.sum().item())
            totals[m]["err_base"] += err_base_count
            totals[m]["err_method"] += int(((~method_correct) & valid).sum().it

    for m in diag_methods:
        valid_n = max(totals[m]["valid"], 1)
        err_base_n = max(totals[m]["err_base"], 1)

        changed_pct = 100.0 * totals[m]["changed"] / valid_n
        good_pct = 100.0 * totals[m]["good"] / valid_n
        bad_pct = 100.0 * totals[m]["bad"] / valid_n
        edit_precision = totals[m]["good"] / max(totals[m]["good"] + totals[m]["bad
        net_gain_pct = good_pct - bad_pct
        error_reduction_pct = 100.0 * (totals[m]["err_base"] - totals[m]["err_metho

        diag_rows.append({
            "distortion": dname,
            "method": m,
            "changed_pct": changed_pct,
            "good_change_pct": good_pct,
            "bad_change_pct": bad_pct,
            "edit_precision": edit_precision,
            "net_gain_pct": net_gain_pct,
            "error_reduction_pct": error_reduction_pct,
```

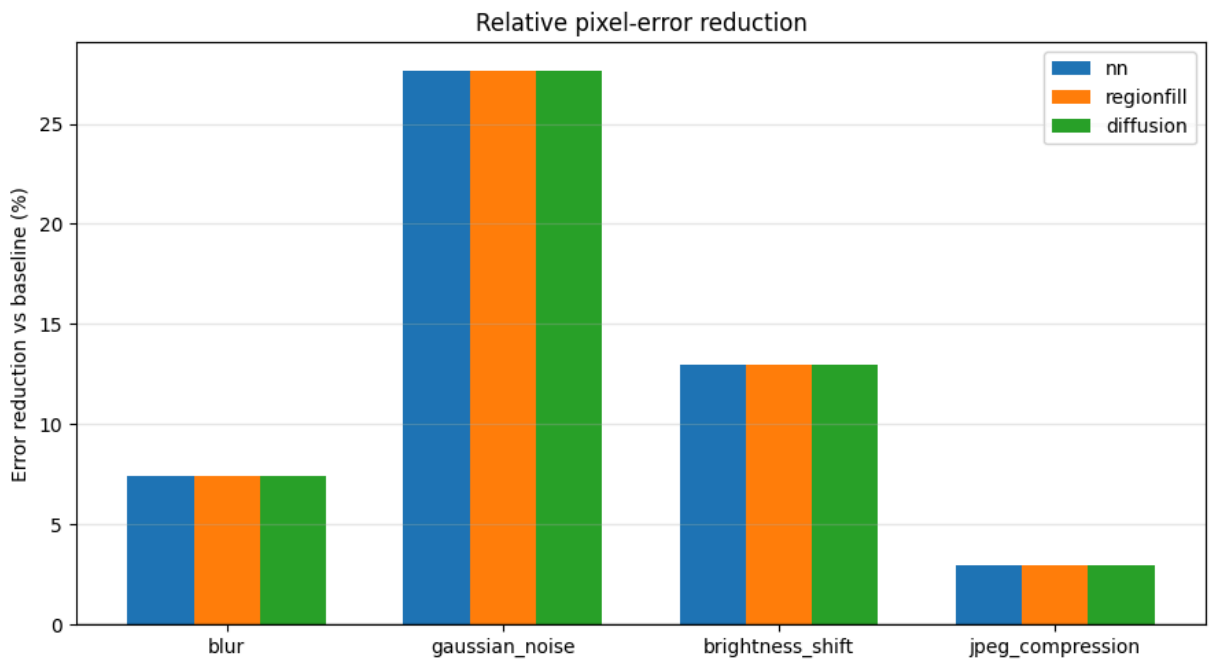
```
        })

df_diag = pd.DataFrame(diag_rows)
display(df_diag.round(4))

df_diag.to_csv(OUT / "change_focused_metrics.csv", index=False)
print("saved:", OUT / "change_focused_metrics.csv")

# Plot 1: net gain by distortion/method
fig, ax = plt.subplots(figsize=(9, 5))
x_dist = np.arange(len(distortions))
w = 0.24
for i, m in enumerate(diag_methods):
    vals = [
        float(
            df_diag[(df_diag["distortion"] == d) & (df_diag["method"] == m)]["net_g
        )
        for d in distortions
    ]
    ax.bar(x_dist + (i - 1) * w, vals, width=w, label=m)
ax.axhline(0.0, color="black", linewidth=1)
ax.set_xticks(x_dist)
ax.set_xticklabels(distortions)
ax.set_ylabel("Net helpful edits (% valid pixels)")
ax.set_title("Net gain from changed pixels (good - bad)")
ax.legend()
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig(OUT / "net_gain_change_focused.png", dpi=200, bbox_inches="tight")
plt.show()

# Plot 2: error reduction by distortion/method
fig, ax = plt.subplots(figsize=(9, 5))
for i, m in enumerate(diag_methods):
    vals = [
        float(
            df_diag[(df_diag["distortion"] == d) & (df_diag["method"] == m)]["error
        )
        for d in distortions
    ]
    ax.bar(x_dist + (i - 1) * w, vals, width=w, label=m)
ax.axhline(0.0, color="black", linewidth=1)
ax.set_xticks(x_dist)
ax.set_xticklabels(distortions)
ax.set_ylabel("Error reduction vs baseline (%)")
ax.set_title("Relative pixel-error reduction")
ax.legend()
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig(OUT / "error_reduction_change_focused.png", dpi=200, bbox_inches="tight
plt.show()

# Compact summary by method across distortions
df_diag_summary = df_diag.groupby("method", as_index=False).agg({
    "changed_pct": "mean",
    "good_change_pct": "mean",
```

```
    "bad_change_pct": "mean",
    "edit_precision": "mean",
    "net_gain_pct": "mean",
    "error_reduction_pct": "mean",
})
display(df_diag_summary.round(4))
df_diag_summary.to_csv(OUT / "change_focused_metrics_summary.csv", index=False)
print("saved:", OUT / "change_focused_metrics_summary.csv")

# Narrative-ready one-liner (truthful):
best = df_diag_summary.sort_values(["net_gain_pct", "error_reduction_pct"], ascendi
print(
    f"Most reliable editor on this setup: {best['method']} "
    f"(avg net_gain={best['net_gain_pct']:.4f}%, avg error_reduction={best['error_r
)
```

	distortion	method	changed_pct	good_change_pct	bad_change_pct	edit_precisio
0	blur	nn	4.5654	3.0256	0.7618	0.798
1	blur	regionfill	4.5632	3.0249	0.7610	0.799
2	blur	diffusion	4.5627	3.0248	0.7608	0.799
3	gaussian_noise	nn	9.0117	6.6391	1.0921	0.858
4	gaussian_noise	regionfill	9.0111	6.6389	1.0918	0.858
5	gaussian_noise	diffusion	9.0103	6.6388	1.0917	0.858
6	brightness_shift	nn	2.1409	1.4851	0.3704	0.800
7	brightness_shift	regionfill	2.1393	1.4846	0.3696	0.800
8	brightness_shift	diffusion	2.1387	1.4844	0.3692	0.800
9	jpeg_compression	nn	7.7315	3.2087	2.6948	0.543
10	jpeg_compression	regionfill	7.7285	3.2076	2.6939	0.543
11	jpeg_compression	diffusion	7.7276	3.2068	2.6939	0.543

```
saved: outputs\change_focused_metrics.csv
```





	method	changed_pct	good_change_pct	bad_change_pct	edit_precision	net_gain_pct	er
0	diffusion	5.8598	3.5887	1.2289	0.7505	2.3598	
1	nn	5.8624	3.5897	1.2298	0.7504	2.3599	
2	regionfill	5.8605	3.5890	1.2291	0.7505	2.3599	

```
saved: outputs\change_focused_metrics_summary.csv
Most reliable editor on this setup: regionfill (avg net_gain=2.3599%, avg error_redu
ction=12.7471%).
```